

Classes, Objects, and Instantiation

Let's see how this works in code.

class

Code that defines what an object will remember (its data or *state*) and the things that it will be able to do (its functions or *behavior*).

To get a feel for what a class looks like, here is the code of a light switch written as a class:

```
# 00_LightSwitch

class LightSwitch():
    def __init__(self):
        self.switchIsOn = False

    def turnOn(self):
        # turn the switch on
        self.switchIsOn = True

    def turnOff(self):
        # turn the switch off
        self.switchIsOn = False
```

We'll go through the details in just a bit, but the things to notice are that this code defines a single variable, `self.switchIsOn`, which is initialized in one function, and contains two other functions for the behaviors: `turnOn()` and `turnoff()`.

If you write the code of a class and try to run it, nothing happens, in the same way as when you run a Python program that consists of only functions and no function calls. You have to explicitly tell Python to make an object from the class.

To create a `LightSwitch` object from our `LightSwitch` class, we typically use a line like this: